Pogovor o kodi in analizi seminarske naloge

Seminarska naloga: hitrost in vrsta meta v MLB · repozitorij JakaRudolf/Projektna-naloga · 20. 10. 2026

> **VPRAŠANJE 1**
>
> *zakaj das v HEADERS User-Agent,kaj je to in kaj bi blo ce ga ne bi dal*

Kaj je User-Agent

Ko tvoj program (ali brskalnik) zahteva spletno stran, strežniku poleg naslova pošlje še nekaj dodatnih podatkov. Tem podatkom rečemo **glave** (angl. *headers*). Ena od njih je **User-Agent**: kratko besedilo, s katerim se program strežniku predstavi, nekako tako kot ime na vizitki.

Ko stran odpreš v Chromu, ta pošlje približno tole:

```
Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/130.0 Safari/537.36
```

Strežnik iz tega razbere, da ga obiskuje Chrome na Windowsu.

Kaj se zgodi, če ga ne podaš

Glava User-Agent se pošlje tudi takrat, ko je sam ne napišeš. Knjižnica `requests` jo v tem primeru doda sama, in sicer takole:

```
python-requests/2.32.3
```

S tem strežniku sporoči, da ga ne obiskuje človek z brskalnikom, ampak Pythonov program. Nekatere strani take obiske zavrnejo, ker se branijo pred programi, ki pobirajo podatke. Možni izidi so:

- strežnik vrne napako, na primer **403 Forbidden** (»dostop prepovedan«),
- strežnik vrne prazno ali drugačno stran od tiste, ki jo vidiš v brskalniku,
- vse deluje čisto normalno, ker stran tega ne preverja.

Zakaj ga torej damo

Gre za previdnost. Ko v `HEADERS` napišemo User-Agent brskalnika, je naša zahteva videti kot običajen obisk, zato je manj verjetno, da nas strežnik zavrne. To je ustaljena praksa pri pobiranju podatkov. Tudi sošolec ga ima v svojem `orodja.py`.

Iskreno povedano, ne vem, ali Baseball Savant tega res ne dovoli. Lahko se zgodi, da bi tvoje skripte delovale tudi brez te glave. Če želiš, lahko to sam preizkusiš v novi celici zvezka:

```
import requests

naslov = "https://baseballsavant.mlb.com/leaderboard/pitch-arsenals?year=2025&min=100&type=avg_speed&hand="

odgovor = requests.get(naslov)
print(odgovor.status_code)
print(len(odgovor.text))
```

Kako prebereš rezultat:

- `status_code` 200 pomeni, da je strežnik zahtevo sprejel, 403 pa, da jo je zavrnil.
- `len(odgovor.text)` pove, koliko znakov je stran dolga. To število primerjaj s tistim, ki ga dobiš, ko v `requests.get` dodaš `headers=HEADERS`. Če sta številki približno enaki, je Savant v obeh primerih vrnil isto stran.

Ko preizkus končaš, celico izbriši, da ne ostane v zvezku.

> **VPRAŠANJE 2**
> *1. kaj sploh nardi raise_for_status()? ce server vrne 404 kaj se pol zgodi v prenesi_stran, kaj vrne ven*

Kaj naredi `raise_for_status()`

Najprej je dobro vedeti, da `requests.get` **sam po sebi ne javi napake**, kadar strežnik vrne 404. Zanj je bil pogovor s strežnikom uspešen: poslal je vprašanje in dobil odgovor. To, da je bil odgovor »te strani ni«, ga ne moti. V `odgovor` tako dobiš čisto običajen odgovor:

- `odgovor.status_code` je `404`,
- `odgovor.text` je HTML strani z napako, nekaj podobnega kot »Page not found«.

`raise_for_status()` je preverjanje, ki ga dodaš sam. Pogleda `status_code`:

- če je koda manjša od 400 (na primer 200, kar pomeni »v redu«), ne naredi ničesar in program teče naprej,
- če je koda 400 ali več (na primer 404 »ni najdeno«, 403 »prepovedano«, 500 »napaka na strežniku«), **sproži napako** (`HTTPError`).

Tvoj komentar »koda stanja = 200« je torej skoraj pravilen. Natančneje bi bilo »koda stanja ni napaka«.

Kaj se zgodi v `prenesi_stran`, če strežnik vrne 404

Po vrsti:

1. `requests.get(...)` se izvede brez težav in v `odgovor` shrani odgovor s kodo 404.
2. `odgovor.raise_for_status()` opazi kodo 404 in sproži napako `HTTPError`.
3. Ker se je napaka zgodila znotraj `try`, Python preostanek bloka preskoči in skoči v `except`. `HTTPError` je ena od vrst `RequestException`, zato jo ta `except` ujame.
4. Izpiše se nekaj takega:

   ```
   spletna stran ni dosegljiva: 404 Client Error: Not Found for url: https://baseballsavant.mlb.com/...
   ```

5. `return None` funkcijo konča. **Funkcija vrne `None`**, torej »nič«.

Vrstica `return odgovor.text` na koncu se v tem primeru sploh ne izvede.

Zakaj je to dobro

Brez `raise_for_status()` bi funkcija vrnila HTML strani z napako, kot da je vse v redu. Skripta bi ga shranila v datoteko in v njem iskala podatke o metih. Ker jih tam ni, bi na koncu dobil prazen CSV in ne bi vedel, zakaj. Tako pa takoj vidiš izpis, da je šlo nekaj narobe in pri katerem naslovu.

Upoštevaj pa še eno stvar. Ko `prenesi_stran` vrne `None`, mora koda, ki jo je poklicala, to obravnavati. Če skripta ta `None` brez preverjanja poda naprej v `shrani_niz_v_datoteko`, se bo program ustavil z napako, ker v datoteko ne more zapisati »nič«. Ker je to napaka, ki jo pričakuješ, s tem ni nič hudega, je pa dobro vedeti, od kod pride.

> **VPRAŠANJE 3**
> *zakaj mas pri zapisi_csv newline=“” pri shrani_niz_v_datoteko pa ne*

Razlika je v tem, **kdo piše konce vrstic**.

Kako Python na Windowsu piše konce vrstic

Konec vrstice je v besedilu nevidni znak. Linux in Mac zanj uporabljata `\n`, Windows pa dva znaka, `\r\n`.

Ko datoteko odpreš z `open(..., "w")`, ti Python pri tem pomaga: na Windowsu vsak `\n`, ki ga zapišeš, **sam pretvori** v `\r\n`. Za običajno besedilo je to ravno prav.

Zakaj to moti pri CSV

Modul `csv` ne zaupa temu, da bo Python pravilno zapisal konce vrstic, zato na konec vsake vrstice sam doda `\r\n`. To ureja standard za CSV. Zdaj pride do težave:

1. `csv` na konec vrstice napiše `\r\n`.
2. Python v tem najde `\n` in ga na Windowsu pretvori v `\r\n`.
3. V datoteki tako pristane `\r\r\n`.

Excel in drugi programi preberejo to kot dva konca vrstice, zato bi med vsakima dvema vrsticama tabele videl **prazno vrstico**.

`newline=""` Pythonu sporoči, naj konce vrstic pusti pri miru in naj ničesar ne pretvarja. Tako `csv` sam poskrbi za konce vrstic in v datoteki ostane natanko en `\r\n`. To je tudi uradno priporočilo v dokumentaciji modula `csv`: pri pisanju vedno uporabi `newline=""`.

Zakaj pri `shrani_niz_v_datoteko` tega ni treba

Tam zapisuješ HTML, ki si ga dobil s strani, in ne uporabljaš modula `csv`. Nihče ne dodaja svojih `\r\n`, zato do podvajanja ne pride. Python bo morda kak `\n` pretvoril v `\r\n`, kar HTML-ju ne škodi. Ko datoteko kasneje prebereš s `preberi_datoteko_v_niz`, Python pretvorbo obrne nazaj, zato dobiš enako besedilo, kot si ga zapisal.

Na kratko: `newline=""` dodaš takrat, ko konce vrstic piše modul `csv`. Za navadno besedilo ga ne potrebuješ.

> **VPRAŠANJE 4**
> *4. kaj ce regex nc ne najde in je ujemanje None, kaj se zgodi, kšno napako dobis*
> *kaj ce regex nc ne najde in je ujemanje None, kaj se zgodi, kšno napako dobis*

Kaj se zgodi v tvoji kodi

V `savant_hitrosti.py` imaš tole:

```
        def izlusci_metalce(vsebina):
            ujemanje = re.search(r"var data = (\[.*?\]);\s*\n", vsebina, flags=re.DOTALL)
            return json.loads(ujemanje.group(1))
```

Če `re.search` vzorca v besedilu ne najde, **ne javi napake**, ampak samo vrne `None`. Prva vrstica se torej izvede brez težav in `ujemanje` je `None`.

Težava nastane v drugi vrstici. `.group(1)` je metoda, ki jo ima le pravo ujemanje, `None` pa je nima. Python se zato ustavi s to napako:

```
AttributeError: 'NoneType' object has no attribute 'group'
```

Prevod: »`None` nima ničesar, kar bi se imenovalo `group`.«

Kaj to pomeni za skripto

Napake nihče ne ujame, ker okoli te kode ni `try`/`except`. Program se zato **takoj ustavi** in izpiše rdeče sporočilo (*traceback*), ki pokaže, v kateri vrstici je prišlo do napake. Vse, kar bi sledilo, se ne izvede: zanka čez metalce se ne zažene in CSV se ne zapiše.

Sporočilo je malo zavajajoče, ker ne pove, da regex ni ničesar našel. Govori o `NoneType` in `group`, zato moraš sam vedeti, da to običajno pomeni »iskanje ni uspelo«. To je ena najpogostejših napak pri delu z regexom.

Kdaj bi se to lahko zgodilo pri tebi

Delno si se pred tem že zaščitil. V `prenos_strani` preverjaš `if "var data = [" not in vsebina`, zato HTML-ja brez podatkov sploh ne shraniš. Regex se vseeno lahko zalomi, na primer:

- če Savant spremeni zapis strani, recimo v `var data=[` brez presledkov, ali če za `];` ni več preloma vrstice, ki ga zahteva `\s*\n`,
- če je bila datoteka shranjena že prej, preden si dodal to preverjanje, saj se ob obstoječi datoteki prenos (in s tem preverjanje) preskoči.

Če bi želel prijaznejše sporočilo, bi lahko pred `return` dodal preprosto preverjanje:

```
            if ujemanje is None:
                print("v HTML-ju ni podatkov o metalcih")
                return []
```

S praznim seznamom zanka v `main` naredi nič korakov in nastane prazen CSV. Ker vidiš izpis, veš, zakaj je prazen. Ni nujno, da to dodaš. Če pustiš kodo, kot je, dobiš pač `AttributeError`.

> **VPRAŠANJE 5**
> *zakaj so tu vrste metov z malimi (“ff”) v rezultatih pa z velkimi (“FF”)? kje to popravs da se pol tabeli zdruzita*

Zakaj sta zapisa različna

Razlika izvira iz **Savanta**. Vsaka od obeh strani vrste metov zapiše po svoje, ti pa se moraš prilagoditi vsaki posebej.

Stran s hitrostmi (`savant_hitrosti.py`): pri tej strani je vrsta meta del **imena ključa** v podatkih. En metalec je en slovar, v njem pa so ključi, kot so:

```
"ff_avg_speed", "si_avg_speed", "cu_avg_spin", "n_ff", ...
```

Ti ključi so pisani z malimi črkami. Zato imaš tudi `VRSTE_METOV` z malimi črkami, saj iz njih sestavljaš imena ključev:

```
        hitrost = v_stevilo(metalec.get(f"{vrsta}_avg_speed"))   # "ff" -> "ff_avg_speed"
```

Če bi namesto tega napisal `"FF"`, bi koda iskala ključ `"FF_avg_speed"`. Takega ključa ni, zato bi `get` vrnil `None` in vse vrstice bi se preskočile.

Stran z rezultati (`savant_rezultati.py`): tu je vrsta meta zapisana v naslovu strani (`pitchType=FF`) in kot **vrednost** v podatkih, v polju `"pitch_type": "FF"`. Savant jo tukaj piše z velikimi črkami, zato imaš `VRSTE_METOV` z velikimi.

Kje se to poenoti

To si že naredil, in sicer v `savant_hitrosti.py`, v funkciji `vrstice_metalca`:

```
        "vrsta_meta": vrsta.upper(),
```

`.upper()` pretvori niz v velike črke: `"ff"` postane `"FF"`. Male črke torej uporabiš le, ko sestavljaš imena ključev, v CSV pa zapišeš velike.

Zato imata oba CSV-ja v stolpcu `vrsta_meta` enake vrednosti (`FF`, `SI`, `CU` …) in se tabeli v zvezku lahko združita po `id_metalca` in `vrsta_meta`. Če `.upper()` ne bi bilo, bi pandas `"ff"` in `"FF"` obravnaval kot različni vrednosti. Pri združevanju se ne bi ujela nobena vrstica in dobil bi prazno tabelo.

Popravljati ti ni treba ničesar, ker je to že urejeno.

> **VPRAŠANJE 6**
> *9. tu mas time.sleep 2-3 sekunde v hitrostih pa ne, zakaj? pa a caka tud ce je stran ze shranjena*

Zakaj je pavza samo v rezultatih

Pavza je zato, da strežnika ne zasipaš z zahtevami eno za drugo. Če program v nekaj sekundah pošlje veliko zahtev, je to za strežnik obremenitev, nekatere strani pa tak promet celo prepoznajo kot napad in te začasno blokirajo. Pavza med zahtevami je zato vljudnost do strani.

- `savant_rezultati.py` prenese **9 strani**, po eno za vsako vrsto meta. Brez pavze bi teh 9 zahtev poslal v zelo kratkem času. Zato po vsaki počaka naključno med 2 in 3 sekundami.
- `savant_hitrosti.py` prenese **samo eno stran**. Pavza je smiselna *med* zahtevami, tukaj pa druge zahteve ni, zato ni ničesar, med čimer bi čakal.

Ali čaka tudi, če je stran že shranjena

Ne. To se vidi iz zaporedja v zanki:

```
        for vrsta in VRSTE_METOV:
            datoteka = ime_datoteke(vrsta)
            if orodja.datoteka_obstaja(MAPA, datoteka):
                continue
            ...
            orodja.shrani_niz_v_datoteko(vsebina, MAPA, datoteka)
            print("shranjeno:", datoteka)
            time.sleep(random.uniform(2, 3))
```

`continue` pomeni: »preskoči preostanek tega kroga zanke in pojdi takoj na naslednjo vrsto«. Če datoteka že obstaja, se zanka vrne na začetek, še preden pride do `time.sleep`. Tak krog ne pošlje nobene zahteve na strežnik, zato tudi čakati ni treba.

Ker imaš vseh 9 strani že shranjenih, se `prenos_strani()` ob ponovnem zagonu konča skoraj v trenutku.

Enako velja za druga dva `continue`, ko prenos ne uspe ali stran nima pravih podatkov: tudi v teh primerih program ne počaka. To je manjša pomanjkljivost, ker je zahteva v teh primerih vseeno šla na strežnik. Za tvojo nalogo to ni pomembno.

> **VPRAŠANJE 7**
>
> ```
> def izlusci_metalce(vsebina):
>     ujemanje = re.search(r"var data = (\[.*?\]);\s*\n", vsebina, flags=re.DOTALL)
>     return json.loads(ujemanje.group(1))
> ```
>
> *ali lahko ta regex naredimo tudi na kaksen drugacen nacin*

Da, več načinov je. Najprej pa kratko, kaj tvoj regex zdaj počne, da se bodo razlike lepo videle.

Kaj počne zdajšnji regex

```
var data = (\[.*?\]);\s*\n
```

1. poišče besedilo `var data =`,
2. nato `[` in začne zajemati,
3. `.*?` jemlje znake, dokler ne pride do prvega mesta, kjer sledi `]`, nato `;`, nato morebitni presledki in prelom vrstice,
4. kar je med oklepajema (skupaj z `[` in `]`), shrani v `group(1)`.

Pomemben je ravno del `;\s*\n`. Iz tvojega komentarja v `savant_rezultati.py` razberem, da se na strani s hitrostmi `];` pojavi tudi *znotraj* podatkov, ne samo na koncu. Če bi se ustavil že pri prvem `];`, bi odrezal podatke na pol. Prelom vrstice za `];` pa se pojavi le na pravem koncu. Na to moraš paziti pri vsaki drugi rešitvi.

1. način: brez regexa, s `find`

```
        def izlusci_metalce(vsebina):
            zacetek = vsebina.find("var data = ") + len("var data = ")
            konec = vsebina.find("];\n", zacetek) + 1
            return json.loads(vsebina[zacetek:konec])
```

- `find` vrne mesto (številko znaka), kjer se besedilo prvič pojavi.
- `zacetek` je mesto takoj za `var data =`, torej tam, kjer stoji `[`.
- `konec` poišče `];\n`, a šele od `zacetek` naprej. `+ 1` je zato, da `]` pride zraven.
- `vsebina[zacetek:konec]` izreže ta kos besedila.

Prednost: vsak korak je preprost in ga lahko izpišeš in preveriš. Slabost: če med `];` in prelomom vrstice stoji presledek, tega ne najde (regex s `\s*` to dopušča). Poleg tega `find` ob neuspehu vrne `-1` in ne javi napake, zato bi dobil napačen izrezek, napaka pa bi se pokazala šele v `json.loads`.

2. način: brez regexa, s `split`

```
        def izlusci_metalce(vsebina):
            za_zacetkom = vsebina.split("var data = ")[1]
            podatki = za_zacetkom.split(";\n")[0]
            return json.loads(podatki)
```

- `split` razreže besedilo na kose povsod, kjer najde dani niz, in vrne seznam kosov.
- `[1]` vzame kos *za* `var data =`.
- Ta kos še enkrat razrežeš pri `;\n` in z `[0]` vzameš, kar je pred tem, torej celoten seznam `[...]`.

Prednost: zelo kratko in berljivo. Slabost: enaka kot pri `find` glede presledkov. Če `var data =` sploh ni, dobiš `IndexError`.

3. način: drugačen regex

Namesto »do `];` s prelomom vrstice« lahko rečeš »celotna vrstica«:

```
        ujemanje = re.search(r"var data = (\[.*\]);", vsebina)
```

Tu ni `?` in ni `re.DOTALL`. Brez `DOTALL` `.` ne prestopi preloma vrstice, zato se `.*` ustavi na koncu vrstice in vzame zadnji `];` v tej vrstici. To deluje le, če so vsi podatki v **eni vrstici** HTML-ja, česar brez pogleda v shranjeno datoteko ne morem zagotovo trditi.

Kaj priporočam

Pusti, kot imaš. Tvoj regex je zanesljivejši od 1. in 2. načina, ker dopušča presledke, in pri predmetu o pridobivanju podatkov je regex verjetno tudi pričakovan. Če želiš kateri od drugih načinov preizkusiti, ga zaženi in preveri, ali `print(len(metalci))` izpiše isto število kot zdaj.

> **VPRAŠANJE 8**
> *kaj tocno naredi dotall*

Kaj pomeni `.` v regexu

V regexu `.` pomeni »katerikoli znak«: črko, številko, presledek, vejico, oklepaj … z eno izjemo. **Prelom vrstice (`\n`) privzeto ni vključen.** Pika se torej ustavi na koncu vrstice in ne more nadaljevati v naslednjo.

`re.DOTALL` to izjemo odstrani. Ko ga dodaš, `.` ujame **res vsak znak, tudi prelom vrstice**. Ime pove ravno to: »dot« (pika) ujame »all« (vse).

Primer

```
        import re

        besedilo = "var data = [1, 2,\n3, 4];\n"

        brez = re.search(r"var data = (\[.*?\]);", besedilo)
        z = re.search(r"var data = (\[.*?\]);", besedilo, flags=re.DOTALL)

        print(brez)
        print(z.group(1))
```

Izpis:

```
None
[1, 2,
3, 4]
```

- **Brez `DOTALL`:** `.*?` pride do konca prve vrstice (`[1, 2,`) in naprej ne more, ker je naslednji znak `\n`. Tam ne najde `];`, zato iskanje ne uspe in vrne `None`.

- **Z `DOTALL`:** `.*?` gre čez prelom vrstice v drugo vrstico in najde `];` za `4`.

Zakaj ga imaš ti

Podatki v HTML-ju (`var data = [ ... ]`) so lahko razpotegnjeni čez več vrstic. `re.DOTALL` poskrbi, da regex najde celoten seznam, tudi če se nadaljuje v naslednje vrstice.

Še ena malenkost: `DOTALL` vpliva **samo na `.`**. Na `\s` in `\n` v tvojem regexu (`;\s*\n`) ne vpliva. `\n` vedno pomeni prelom vrstice, `\s` pa vsak »prazen« znak, med katere spada tudi prelom vrstice, ne glede na to, ali `DOTALL` uporabiš ali ne.

> **VPRAŠANJE 9**
> *zakaj preverjas da je "pitchType":"{vrsta}" v vsebini, pred cim te to sciti*

To preverjanje varuje pred napako, ki jo je težko opaziti: **strežnik vrne pravo stran s podatki, a za napačno vrsto meta.**

Kaj preverita prejšnja dva `if`-a

Pred tem že preveriš:

1. `vsebina is None`: ali je prenos sploh uspel,
2. `"var leaderboardData = [{" not in vsebina`: ali stran vsebuje podatke.

Nobeno od teh preverjanj ne pove, **katere** podatke si dobil. Stran z metalci in številkami bi prestala obe, četudi bi šlo za druge mete.

Kdaj bi to šlo narobe

Do tega lahko pride, če strežnik vrednosti v naslovu ne upošteva. Če `pitchType=SV` ne prepozna (na primer zaradi tipkarske napake v `VRSTE_METOV` ali ker bi Savant to vrsto preimenoval), ne javi napake. Namesto tega vrne privzeto stran, na primer za hitre mete (FF). Strani, ki delujejo s filtri, se v takem primeru pogosto obnašajo tako.

Brez tega preverjanja bi se zgodilo tole:

- podatke za FF bi shranil v datoteko `stran_savant_rezultati_SV_2025.html`,
- v CSV bi jih zapisal z vrsto meta `SV`, ker jo `pocisti_vrstico` vzame iz podatkov (`zapis["pitch_type"]`), ali pa bi dobil podvojene vrstice za FF,
- program bi se končal brez napake in s številkami, ki so videti povsem normalne.

Težava je še večja, ker datoteke ne preneseš ponovno, če že obstaja. Napačna datoteka bi zato ostala v mapi pri vsakem naslednjem zagonu.

Kako preverjanje deluje

Savant v HTML strani zapiše tudi, kateri filter je izbran, nekako takole:

```
"pitchType":"SV"
```

S tem `if`-om preveriš, ali stran res pravi, da gre za vrsto, ki si jo zahteval:

```
        if f'"pitchType":"{vrsta}"' not in vsebina:
            print("stran ni za vrsto meta", vrsta)
            continue
```

Če tega zapisa ni, se stran ne shrani in v izpisu vidiš, pri kateri vrsti meta je šlo kaj narobe.

Pri `savant_hitrosti.py` takega preverjanja ni, ker tam preneseš eno samo stran, ki vsebuje vse vrste metov hkrati. Nobene vrste ni mogoče pomešati z drugo.

> **VPRAŠANJE 10**
> *zakaj je STOLPCI tu slovar v hitrostih pa seznam? kaj je kljuc kaj vrednost*

Kaj je ključ in kaj vrednost v `savant_rezultati.py`

```
        STOLPCI = {
            "id_metalca": "player_id",
            "vrsta_meta": "pitch_type",
            "whiff": "whiff_percent",
            ...
        }
```

- **Ključ** (levo) je ime stolpca v **tvojem CSV-ju**, po slovensko.
- **Vrednost** (desno) je ime polja v **Savantovih podatkih**, po angleško.

Slovar je torej nekakšen prevajalnik: »stolpec `whiff` napolni s tem, kar Savant imenuje `whiff_percent`«.

Uporabiš ga na dveh mestih:

```
        for stolpec, polje in STOLPCI.items():
            ...
            vrstica[stolpec] = v_stevilo(zapis[polje])
```

`.items()` da v vsakem koraku zanke en par, na primer `stolpec = "whiff"` in `polje = "whiff_percent"`. Koda vzame vrednost iz Savantovega zapisa pod angleškim imenom in jo shrani pod slovenskim.

```
        orodja.zapisi_csv(list(STOLPCI.keys()), vrstice, MAPA, CSV_DATOTEKA)
```

Tu potrebuješ samo slovenska imena za glavo CSV-ja, zato vzameš le ključe.

To deluje, ker je pri rezultatih povezava vedno enaka: `whiff` je vedno `whiff_percent`, ne glede na vrsto meta.

Zakaj v `savant_hitrosti.py` to ne gre

Pri hitrostih so imena polj v Savantu **odvisna od vrste meta**:

```
"ff_avg_speed", "si_avg_speed", "cu_avg_speed", ...
```

Stolpcu `hitrost` zato ne moreš pripisati enega samega Savantovega imena, ker je to za vsako vrsto drugo. Prevajanje zato poteka ročno v `vrstice_metalca`, kjer ime sestaviš sproti:

```
        "hitrost": hitrost,  # iz metalec.get(f"{vrsta}_avg_speed")
        "vrtenje": v_stevilo(metalec.get(f"{vrsta}_avg_spin")),
```

Tam iz enega metalca narediš več vrstic, po eno za vsako vrsto meta.

`STOLPCI` je v `savant_hitrosti.py` zato le **seznam imen stolpcev**. Potrebuješ ga samo za to, da `zapisi_csv` ve, katere stolpce in v kakšnem vrstnem redu naj zapiše v glavo CSV-ja.

Na kratko: pri rezultatih je slovar, ker hkrati določa imena stolpcev in pove, od kod jih napolniti. Pri hitrostih se polnjenje dogaja drugje, zato zadošča seznam.

> **VPRAŠANJE 11**
> *13. zakaj ma zdruzena tabela 558 vrstic, ne tok kot hitrosti al rezultati? pa kaj bi dobu ce bi dal how="left"*

Zvezka na GitHubu ne morem odpreti (ne prikaže se mi), zato bom razložil splošno. Predpostavljam, da tabeli združuješ približno tako:

```
        zdruzeno = pd.merge(hitrosti, rezultati, on=["id_metalca", "vrsta_meta"])
```

Zakaj 558 vrstic

Če `how` ne navedeš, pandas uporabi `how="inner"`. To pomeni, da v združeni tabeli ostane **samo tisti par metalec × vrsta**

meta, ki obstaja v obeh tabelah. Vrstica, ki je samo v hitrostih ali samo v rezultatih, izpade.

Tabeli se razlikujeta zaradi meje `min=100` v naslovih, ki pri vsaki strani pomeni nekaj drugega:

- **Hitrosti**: meja velja za metalca kot celoto. Če je metalec prišel čez mejo, so zraven vse njegove vrste metov, tudi tista, ki jo je vrgel le redko. Taka vrsta ima hitrost, rezultatov zanjo pa ni dovolj.
- **Rezultati**: meja velja za **vsako vrsto meta posebej**. Redko uporabljene vrste se v tej tabeli ne pojavijo.

Posledično imajo hitrosti veliko parov, ki jih v rezultatih ni. Velja pa lahko tudi obratno: v rezultatih je lahko kak par, ki ga v hitrostih ni. Če ima metalec premalo metov skupaj, ga v hitrostih sploh ni, a lahko ima pri eni vrsti dovolj rezultatov. Pri `inner` izpadejo oboji, zato je 558 manj od števila vrstic v obeh tabelah.

Kaj bi dobil s `how="left"`

```
        zdruzeno = pd.merge(hitrosti, rezultati, on=["id_metalca", "vrsta_meta"], how="left")
```

»Left« pomeni, da ohraniš **vse vrstice leve tabele**, to je tiste, ki je v `merge` napisana prva (tu `hitrosti`):

- kjer se par ujema, dobiš vrstico z vsemi stolpci, kot zdaj,
- kjer ujemanja v rezultatih ni, vrstica vseeno ostane, stolpci iz rezultatov (`whiff`, `hitrost_odboja` …) pa so prazni oziroma `NaN`,
- pari, ki so samo v rezultatih, izpadejo.

Združena tabela bi imela **natanko toliko vrstic kot** `hitrosti`. Za analizo te dodatne vrstice ne bi bile koristne, ker nimajo rezultatov. Zato je `inner` za tvojo nalogo prava izbira.

Če želiš to preveriti

V novi celici zvezka:

```
        print(len(hitrosti))
        print(len(rezultati))

        levo = pd.merge(hitrosti, rezultati, on=["id_metalca", "vrsta_meta"], how="left")
        print(len(levo))
        print(levo["whiff"].isna().sum())
```

Zadnja vrstica prešteje, koliko vrstic nima ujemanja v rezultatih. Ker je `len(levo)` enak številu vrstic v hitrostih (558 ujemajočih se plus te brez ujemanja), mora veljati: `len(levo)` minus ta številka da 558. (Natančno to drži, če `whiff` v rezultatih nikjer ne manjka sam od sebe, kar je zelo verjetno.)

Ko končaš, celico izbriši.

> **VPRAŠANJE 12**
> *kako do da zvezka ne mores odpreti? zahtevaj dostop od mene se enkrat*

Dostopa ti ni treba dajati, repozitorij je javen. Težava je bila v tem, kako GitHub prikazuje zvezke: stran za `.ipynb` ga ne prikaže neposredno, ampak ga naknadno naloži z JavaScriptom. Orodje, s katerim berem spletne strani, JavaScripta ne izvaja, zato je dobilo le prazen okvir strani brez vsebine zvezka. Zdaj sem repozitorij namesto tega prenesel z `git clone`, tako kot ti, in zvezek prebral.

Pri tem sem ugotovil, da moj prejšnji odgovor ni bil čisto pravilen. Tvoje številke so:

```
hitrosti: (3412, 10)
rezultati: (558, 13)
združeno: (558, 21)
```

Zakaj ima združena tabela 558 vrstic

Združena tabela ima **natanko toliko vrstic kot** `rezultati`. To pomeni, da se je **vsak** par metalec × vrsta meta iz rezultatov našel tudi v hitrostih. V rezultatih ni para, ki ga ne bi bilo v hitrostih, zato iz te tabele pri `inner` nič ne izpade.

Izpade pa veliko vrstic iz hitrosti: 3412 − 558 = **2854**. Razlog je, kot sem pisal, različna meja `min=100`:

- v **hitrostih** je vsaka vrsta meta vsakega metalca, ki je dosegel mejo, tudi če jo je vrgel le nekajkrat,
- v **rezultatih** je samo par, ki ima pri tej vrsti meta dovolj rezultatov.

Rezultati so torej strožji filter in so v celoti »vsebovani« v hitrostih. `inner` obdrži le preseke, zato ostane 558.

Kaj bi dobil s `how="left"`

```
pd.merge(hitrosti, rezultati, on=["id_metalca", "vrsta_meta"], how="left")
```

Leva tabela je `hitrosti`, zato bi dobil **3412 vrstic**:

- 558 bi jih bilo enakih kot zdaj, z vsemi podatki,
- 2854 bi jih imelo hitrost, vrtenje in gibanje, stolpci iz rezultatov (`whiff`, `hitrost_odboja`, `ba` ...) pa bi bili `NaN`.

Za tvojo analizo so te dodatne vrstice neuporabne, ker potrebuješ ravno povezavo med hitrostjo in rezultatom. Zato je `inner` pravilna izbira.

Zanimivost: če bi dal `how="right"` (ohrani vse vrstice desne tabele, `rezultati`), bi dobil **isto kot z `inner`**, torej 558 vrstic. To je še en dokaz, da se vsaka vrstica iz rezultatov ujema s kakšno vrstico iz hitrosti.

> **VPRAŠANJE 13**
>
> *zakaj mas pri stolpicnem grafu ylim(70, 100)? a ne bi to koga lahk zavedl*

Da, imaš prav. To je znan problem pri stolpčnih grafih.

Zakaj je to lahko zavajajoče

Pri stolpčnem grafu bralec primerja **dolžine stolpcev**. Oko nehote sklepa: »ta stolpec je dvakrat višji, torej je vrednost dvakrat večja.« To pa drži samo, če se os začne pri 0.

Pri tebi se začne pri 70. Vzemimo FF (94,6 mph) in CU (80,6 mph):

- dejansko razmerje: 94,6 / 80,6 ≈ **1,17**, FF je torej za približno 17 % hitrejši,
- višini stolpcev nad 70: FF 24,6, CU 10,6, razmerje ≈ **2,3**. Na grafu je FF videti več kot dvakrat hitrejši.

Razlika je resnična, graf pa jo prikaže precej večjo, kot je.

Zakaj je bil `ylim(70, 100)` sploh tam

Vse povprečne hitrosti so med približno 80 in 95 mph. Če se os začne pri 0, so vsi stolpci skoraj enako visoki in razlike med vrstami se komaj vidijo. `ylim` je razlike »povečal«, s tem pa tudi popačil razmerja.

Kaj lahko narediš

1. Odstraniš `plt.ylim(70, 100)`. Os se začne pri 0, graf je pošten, razlike pa so videti majhne. To je tudi resnica: v primerjavi s celotno hitrostjo so razlike res sorazmerno majhne.

2. Namesto stolpcev uporabiš pike. Pri piki oko ne primerja dolžine, ampak samo položaj na osi, zato se os sme začeti kjerkoli:

```
plt.plot(povprecja.index, povprecja["hitrost"], "o")
plt.title("Povprečna hitrost meta po vrstah metov")
plt.xlabel("Vrsta meta")
plt.ylabel("Hitrost (mph)")
plt.show()
```

`"o"` pomeni »nariši samo kroge, brez črt med njimi«. Os y si matplotlib nastavi sam, okoli dejanskih vrednosti, in razlike so dobro vidne brez zavajanja. Črte med točkami ne bi imele smisla, ker vrste metov niso zaporedne (med FF in SI ni ničesar »vmes«).

Jaz bi izbral 2. način. Če ti je ljubši stolpčni graf, izberi 1. Pri drugem grafu (whiff) `ylim` nimaš, zato je tam vse v redu.

Ko spremeniš celico, ne pozabi: **Restart + Run All → Ctrl+S**, nato git.

> **VPRAŠANJE 14**
> *vsaka pika na grafu je en metalec z eno vrsto meta, tist ki je vrgu 150 metov pa tist k jih je 1500 stejeta isto. a je to problem za korelacijo*

Da, pomislek je upravičen. Preveril sem ga kar na tvojih podatkih.

Zakaj bi to lahko bil problem

Odstotek, izračunan iz malo metov, je bolj **naključen** od odstotka iz veliko metov. Kot pri metu kovanca: po 10 metih imaš lahko 70 % grbov po čistem naključju, po 1000 metih pa boš skoraj gotovo blizu 50 %.

Metalec s 300 meti ima zato whiff %, v katerem je več »šuma« kot pri metalcu s 1500 meti. Pri navadni korelaciji pa obe piki štejeta enako.

Kaj to naredi korelaciji: šum običajno **ne ustvari** povezave, ki je ni. Pike pa raztrese naključno gor in dol, zato je obstoječa povezava videti **šibkejša**, kot je v resnici.

Kaj pokažejo tvoji podatki

Pri FF ima najmanjši metalec v tabeli **296 metov**, ne 150. Mediana je 630, največ pa 1651. Korelacijo sem izračunal pri različnih spodnjih mejah:

Najmanj metov	Metalcev	hitrost–whiff	hitrost–hitrost odboja
vsi	208	0,36	0,05
500	151	0,50	−0,04
800	65	0,49	−0,21
1000	40	0,52	−0,29

Hitrost–whiff: ko izločiš metalce z manj meti, se korelacija okrepi z 0,36 na približno 0,5 in tam ostane. To se ujema z razlago o šumu: povezava je resnična in nekoliko močnejša, kot kaže tvoja številka 0,36.

Hitrost–hitrost odboja: pri vseh metalcih je korelacija praktično 0, pri strožjih mejah pa postaja negativna. Tu bodi previden: pri 65 ali 40 metalcih je vzorec že majhen in tudi korelacija je zato nezanesljiva. Iz tega ne bi sklepal ničesar trdnega.

Kaj lahko narediš v nalogi

Najpreprosteje je, da v zvezek dodaš celico, ki pokaže, da si na to pomislil:

```
ff_vec = ff[ff["meti"] >= 500]
print("število FF z vsaj 500 meti:", len(ff_vec))
print("korelacija:", round(ff_vec["hitrost"].corr(ff_vec["whiff"]), 2))
```

V besedilu nato napišeš eno ali dve povedi: da imajo metalci z malo meti bolj naključne odstotke in da se korelacija med hitrostjo in whiffom pri metalcih z vsaj 500 meti okrepi na 0,5.

Obstaja tudi **utežena korelacija**, pri kateri pika metalca s 1500 meti šteje več kot pika metalca s 300 meti (pri tebi bi dala 0,40). Za začetnika je zapletenejša, ker je pandas nima kot preprosto funkcijo, zato bi ostal pri meji 500.

> **VPRAŠANJE 15**
> *vsi meti skupi je hitrost–whiff −0,53, sam FF pa +0,36. kako lahk ista dva stolpca dasta ravno obratn znak? razlozi po svoje*

Gre za znan pojav, ki se imenuje **Simpsonov paradoks**: povezava, ki velja znotraj vsake skupine, se lahko obrne, ko skupine združiš. Pri tebi se vrste metov med seboj razlikujejo bolj kot metalci znotraj iste vrste.

Dve različni vprašanji

Čeprav uporabljaš ista dva stolpca, korelaciji odgovarjata na različni vprašanji.

Vse vrste skupaj (−0,53) v resnici sprašuje: *ali imajo počasnejše vrste metov več zgrešenih zamahov?*

Odgovor je da, ker so počasni meti drugačni meti. Krivulje, sliderji in changeupi so počasni ravno zato, ker so narejeni za to, da se ukrivijo ali zavedejo batterja s spremembo hitrosti. Njihov namen je, da batter zgreši. Hitri meti (FF, SI) so precej bolj »naravnost«:

Vrsta	Hitrost	Whiff %
FF	94,6	21,4
SI	93,9	13,4
SL	86,3	34,4
CH	85,4	31,8
CU	80,6	35,0

Hitri meti imajo v povprečju malo zgrešenih zamahov, počasni pa veliko. Ko vse pike narišeš skupaj, dobiš oblak, ki pada z leve navzgor proti desni navzdol, zato je korelacija negativna.

Samo FF (+0,36) sprašuje nekaj drugega: *če vsi mečejo isto vrsto meta, ali hitrejši metalci dosežejo več zgrešenih zamahov?*

Tu se vrsta meta ne spreminja, zato ostane samo hitrost. Hitrejši fastball pride do batterja prej, ta ima manj časa za odločitev in pogosteje zgreši. Zato je korelacija pozitivna.

Primerjava iz vsakdanjega življenja

Vzemimo vozila in porabo goriva. Če primerjaš vse skupaj (kolesa z motorjem, avtomobile, tovornjake), imajo težja vozila večjo porabo. Če pa primerjaš samo tovornjake, lahko ugotoviš, da imajo novejši, težji tovornjaki *manjšo* porabo, ker imajo boljše motorje. Obe ugotovitvi držita, ker primerjaš različne stvari.

Kaj pokažejo druge vrste

Preveril sem korelacijo hitrost–whiff znotraj vsake vrste posebej:

Vrsta	Metalcev	Korelacija
FF	208	+0,36
CU	34	+0,34
SI	100	+0,27
ST	35	+0,20
SL	76	+0,10
FC	32	+0,01
CH	54	−0,18
FS	17	−0,51

Pri večini vrst je korelacija pozitivna, kot pri FF, ni pa povsod. Pri changeupu (CH) in splitterju (FS) je negativna. To je smiselno: ta dva meta delujeta tako, da sta *počasnejša* od fastballa, ki ga metalec sicer meče, zato se batter zmoti zaradi razlike v hitrosti. Pri FS je metalcev samo 17, zato ta številka ni zanesljiva.

Kaj to pomeni za tvojo nalogo

Številka −0,53 ne pomeni, da hitrost škodi. Pove le, da se vrste metov med seboj zelo razlikujejo. Če želiš vedeti, kako hitrost vpliva na izid, moraš primerjati znotraj iste vrste meta, kot si to naredil pri FF. To je dobra ugotovitev za v nalogo, saj pokaže, da si razumel, zakaj se ti številki razlikujeta.

> **VPRAŠANJE 16**
> *vsi meti skupi je hitrost–whiff −0,53, sam FF pa +0,36. kako lahk ista dva stolpca dasta ravno obratn znak?*

To je isto vprašanje kot prej, zato tokrat razlaga na drug način, s čisto majhnim izmišljenim primerom.

Primer s štirimi pikami

Recimo, da imaš samo dva metalca in dve vrsti meta:

	Vrsta	Hitrost	Whiff %
A	FF	93	18
B	FF	97	24
A	CU	78	32
B	CU	82	38

Znotraj FF: B je hitrejši (97 proti 93) in ima več whiffov (24 proti 18). Hitreje pomeni več, zato je povezava **pozitivna**.

Znotraj CU: B je hitrejši (82 proti 78) in ima več whiffov (38 proti 32). Spet hitreje pomeni več, povezava je **pozitivna**.

Vse štiri pike skupaj: obe krivulji (78, 82) imata visok whiff (32, 38), oba fastballa (93, 97) pa nizkega (18, 24). Če gledaš vse štiri pike hkrati, se počasnejše pike nahajajo zgoraj, hitrejše pa spodaj. Povezava je zato **negativna**.

Na sliki bi to izgledalo kot dve kratki črti, ki se vzpenjata, vendar je ena (CU) levo zgoraj, druga (FF) pa desno spodaj. Vsaka črta zase gre navzgor, ko pa ju gledaš skupaj, gre navzdol.

Bistvo

Korelacija vseh metov skupaj meri predvsem **razliko med vrstami**: počasne vrste so narejene za zgrešene zamahe. Korelacija samo FF pa meri **razliko med metalci pri isti vrsti**: hitrejši fastball je težje zadeti.

Številki si zato ne nasprotujeta. Odgovarjata na dve različni vprašanji.

> **VPRAŠANJE 17**
> *pri FF je hitrost–hitrost odboja 0,05. a to pomen da hitrost fastballa nc ne vpliva na to kok mocno ga odbijejo*

Ne povsem. Korelacija 0,05 pove, da **v tvojih podatkih ni vidne povezave**. To pa še ne pomeni, da hitrost nima nobenega vpliva. Tak rezultat lahko nastane iz več razlogov.

1. Dva učinka, ki se izničita

Hitrost fastballa vpliva na odboj na dva nasprotna načina:

- **Fizika:** hitrejša žoga ima več energije. Če jo batter zadene enako dobro, odleti z nekoliko večjo hitrostjo. Hitrejši met → močnejši odboj.
- **Čas za odziv:** pri hitrejši žogi ima batter manj časa, zato jo težje zadene točno v sredino palice. Pogosteje jo zadene z robom ali prepozno. Hitrejši met → šibkejši odboj.

Če sta oba učinka podobno močna, se v povprečju izničita in korelacija je blizu 0, čeprav hitrost na odboj vpliva, in to dvakrat.

2. Hitrost odboja vidiš le, ko batter žogo zadene

`hitrost_odboja` se meri samo pri metih, ki jih batter spravi v igro. Zgrešeni zamahi v to povprečje ne štejejo. Pri hitrejših fastballih pa so zgrešeni zamahi pogostejši (to je tvoja korelacija 0,36 z whiffom).

Učinek hitrosti se torej deloma pokaže v tem, *ali* batter žogo zadene, ne v tem, *kako močno*. Tvoja hitrost odboja kaže le tisti del metov, ki jih je batter uspel zadeti.

3. Drugi dejavniki so pomembnejši

Kako močno je žoga odbita, je odvisno tudi od tega, kam je met letel (sredina ali rob cone), kako se je gibal in kdo je bil

batter. Razlike v hitrosti med metalci pri FF so razmeroma majhne, večinoma nekaj mph okoli 95. Ti drugi dejavniki lahko vpliv hitrosti povsem prekrijejo.

Kaj lahko zapišeš v nalogo

Pravilno bi bilo nekaj takega: *»Pri FF med povprečno hitrostjo meta in povprečno hitrostjo odboja ni opazne linearne povezave (r = 0,05). Hitrost meta se pozna predvsem pri tem, ali batter žogo sploh zadene (whiff), ne pa pri tem, kako močno jo odbije, ko jo zadene.«*

S tem ne trdiš, da hitrost nima vpliva, ampak le opišeš, kaj podatki pokažejo.

> **VPRAŠANJE 18**
>
> *ta pogovor spravi v dokument ki ga lahko spravim na racunalnik*

Ogrodje dokumenta je pripravljeno, zdaj pišem prvo poglavje.

Dokument z vprašanji o kodi in analizi je pripravljen. Na računalnik ga shraniš z izvozom v Word ali PDF, lahko pa ga tudi urejaš ali komentiraš.